

# Ecosistema de automatización para Arrecife

Un restaurante de La Paloma recibe pedidos y reservas por su web. El sistema los valida, los razona contra las políticas del local con un modelo de lenguaje, los registra en Notion y avisa al dueño — deteniéndose a pedir aprobación humana sólo cuando la reserva lo amerita.

|                   |                                                                                           |
|-------------------|-------------------------------------------------------------------------------------------|
| CASO DE USO       | Triage automático de reservas de restaurante                                              |
| ORQUESTADOR       | n8n Cloud                                                                                 |
| BASE DE DATOS     | Notion — 5 bases relacionadas                                                             |
| PROCESAMIENTO IA  | Anthropic Claude (RAG sobre políticas + salida estructurada)                              |
| CANALES DE SALIDA | Gmail al cliente · Telegram al encargado                                                  |
| APLICACIÓN        | React + funciones serverless en Vercel                                                    |
| REPOSITORIO       | <a href="https://github.com/MauricioRubino/ArreApp">github.com/MauricioRubino/ArreApp</a> |
| PRODUCCIÓN        | <a href="https://arre-app.vercel.app">arre-app.vercel.app</a>                             |

## CONTENIDO

- 1 · Mapa de arquitectura

2 · Estructuras de datos

3 · Optimización de costos
- 4 · Seguridad y resiliencia

5 · Dashboard de control

6 · Evidencia de pruebas

# 1 · Mapa de arquitectura

El sistema resuelve dos procesos de negocio distintos que comparten la misma puerta de entrada. La diferencia entre ellos no es cosmética: define quién es dueño del dato.

**Decisión central.** En los pedidos, la aplicación escribe en Notion y n8n sólo avisa. En las reservas, *n8n es quien escribe*, porque el estado de una reserva depende de un análisis que ocurre dentro del flujo. Confundir los dos repartos rompe cualquiera de los dos procesos.

## 1.1 · Flujo de pedidos

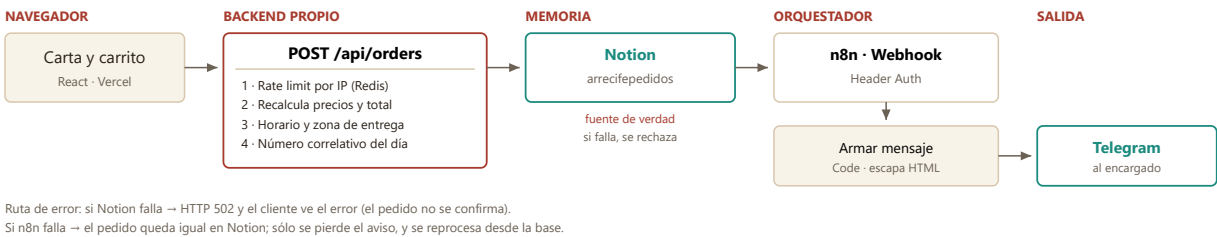


Figura 1 — Pedidos. La aplicación es dueña de la escritura; n8n es un ramal de notificación.

## 1.2 · Flujo de reservas

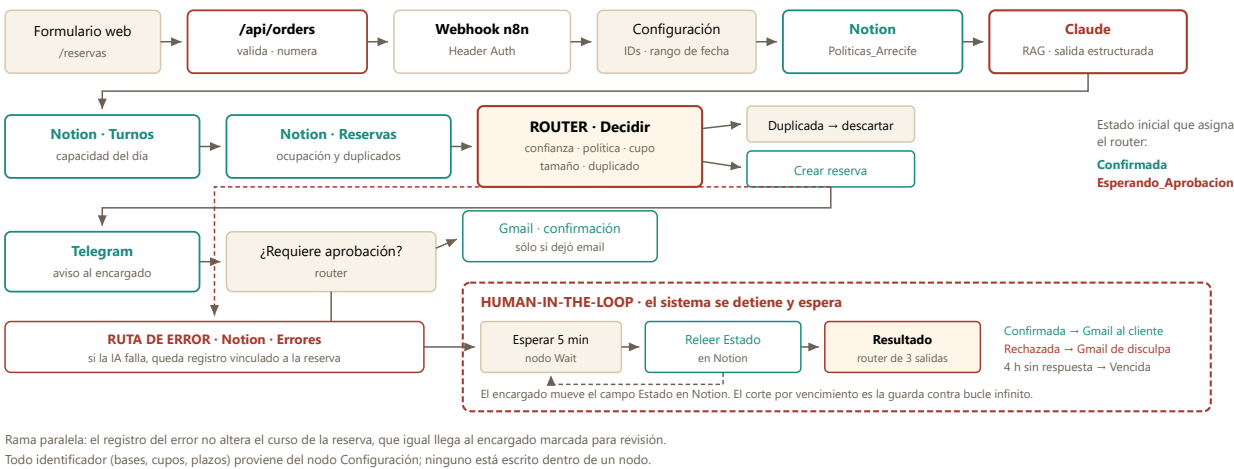


Figura 2 — Reservas. 28 nodos: trigger, dos routers, cuatro APIs externas, un nodo de IA, ruta de error y punto de aprobación humana.

## 1.3 · Disparadores

| PROCESO           | DISPARADOR                                     | POR QUÉ                                                                                                                                                                                                                     |
|-------------------|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pedido nuevo      | Webhook POST<br>/webhook/arrecife-<br>pedidos  | Webhook y no sondeo. Un trigger que consulta una hoja o un buzón cada minuto gasta operaciones las 24 horas para detectar unas pocas decenas de eventos diarios. El webhook se ejecuta exactamente una vez por evento real. |
| Reserva nueva     | Webhook POST<br>/webhook/arrecife-<br>reservas |                                                                                                                                                                                                                             |
| Aprobación humana | Nodo Wait de 5 minutos, reintentando           | Notion no emite webhooks de cambios. El sondeo se acota a la reserva concreta que está esperando y muere a las 4 horas.                                                                                                     |

## 2 · Estructuras de datos

Cinco bases en Notion, con relaciones explícitas para que ningún registro quede aislado. Los tipos importan: la API rechaza la escritura si el nombre o el tipo de una propiedad no coincide exactamente.

### 2.1 · Modelo de datos



Figura 3 — Las cinco bases. Las líneas llenas son relaciones de Notion; la punteada es lectura sin relación.

**Campos de estado.** Reservas.Estado es el eje del sistema: Esperando\_Aprobacion → Confirmada / Rechazada / Vencida . Es lo que escribe el router, lo que sondea el bucle de aprobación y lo que el encargado modifica a mano para dar su veredicto. Los pedidos usan el suyo: Nuevo → Preparando → Listo → En camino → Entregado .

### 2.2 · Esquemas JSON de transferencia

#### A · APLICACIÓN → N8N (RESERVA NUEVA)

Los datos ya vienen validados y normalizados: el flujo no vuelve a chequear nada. El teléfono llega en E.164 y las fechas con el desfase horario explícito.

```
POST /webhook/arrecife-reservas
X-Arrecife-Secret: <secreto compartido>

{
  "tipo": "reservation",
  "numero": 7,
  "fecha": "2026-08-23",
  "recibido": "2026-08-23T18:04:13-03:00",
  "datos": {
    "nombre": "Carolina Méndez",
    "telefono": "+59899202020",
    "email": "carolina@ejemplo.com",
    "fecha": "2028-04-22",
    "fechaLabel": "Sábado, 22 de abril",
    "hora": "21:00",
    "fechaHora": "2028-04-22T21:00:00-03:00",
    "personas": 6,
    "zona": "terrazza",
    "zonaLabel": "Terraza frente al puerto",
    "comentario": "Festejamos un aniversario..."
  }
}
```

## B · N8N → ANTHROPIC (ANÁLISIS CON RAG)

El cuerpo se arma en un nodo *Code*, no en una expresión: así puede ejecutarse y probarse como JavaScript común. El *system prompt* es dinámico — se rearma en cada ejecución con las políticas vigentes.

```
{
  "model": "claude-opus-5",
  "max_tokens": 4000,
  "system": "Sos un asistente que evalúa solicitudes de reserva...\n\n
    POLÍTICAS VIGENTES DE ARRECIFE:\n
    - [Menu/Media] Descorche de bebidas propias: ...\n
    - [Politica_Reserva/Alta] Mascotas en el salón: ...\n
    ... 23 políticas recuperadas de Notion",
  "output_config": {
    "effort": "low",
    "format": {
      "type": "json_schema",
      "schema": {
        "type": "object",
        "properties": {
          "requiere_revision_humana": { "type": "boolean" },
          "motivo_revision": { "type": ["string", "null"] },
          "confianza": { "type": "number" },
          "viola_politica": { "type": "boolean" },
          "politica_relacionada": { "type": ["string", "null"] }
        },
        "required": ["requiere_revision_humana", "motivo_revision",
          "confianza", "viola_politica", "politica_relacionada"],
        "additionalProperties": false
      }
    },
  },
  "messages": [{ "role": "user",
    "content": "Nombre: ...\nPersonas: 6\nFecha: ...\nComentario: ..." }]
}
```

**Mapeo de la respuesta.** El JSON *no* está en `content[0].text`. El modelo razona antes de responder, así que el primer bloque es de tipo `thinking` y el texto viene después. Se busca el bloque por tipo: `content.find(b => b.type === 'text')`. Asumir la posición hacía que el análisis fallara en silencio en todas las ejecuciones.

## C · N8N → NOTION (ALTA DE LA RESERVA)

```
POST https://api.notion.com/v1/pages
Notion-Version: 2022-06-28
```

```
{
  "parent": { "database_id": "{ Configuración.bases.reservas }" },
  "properties": {
    "Nombre_Cliente": { "title": [{ "text": { "content": "Carolina Méndez" }}] },
    "Numero": { "number": 7 },
    "Estado": { "select": { "name": "Esperando_Aprobacion" }},
    "Canal_Origen": { "select": { "name": "Web" }},
    "Telefono": { "rich_text": [{ "text": { "content": "+59899202020" }}] },
    "Email": { "email": "carolina@ejemplo.com" },
    "Fecha": { "date": { "start": "2028-04-22T21:00:00-03:00" }},
    "Personas": { "number": 6 },
    "Zona": { "select": { "name": "Terraza frente al puerto" }},
    "Turno": { "relation": [{ "id": "<página del turno>" }] },
    "Confianza_IA": { "number": 0.94 },
    "Requiere_Revision": { "checkbox": true },
    "Motivo_Revision": { "rich_text": [{ "text": { "content": "..." }}] }
  }
}
```

## D · CONSULTA DE OCUPACIÓN

Se filtra por *rango* y no por igualdad de fecha. Notion compara en UTC: una reserva de las 21:00 -03:00 queda registrada como el día siguiente en UTC, de modo que `{ equals: "2028-04-22" }` no devuelve ninguna reserva de la cena — y el turno de cena va de 19:30 a 00:00.

```
{ "filter": { "and": [
  { "property": "Fecha", "date": { "on_or_after": "2028-04-22T00:00:00-03:00" }},
  { "property": "Fecha", "date": { "before": "2028-04-23T00:00:00-03:00" }},
  { "property": "Estado", "select": { "equals": "Confirmada" }}
]}, "page_size": 100 }
```

### 3 · Optimización de costos

#### 3.1 · Qué tareas usan IA, y cuáles no

La primera decisión de costo es no usar un modelo donde alcanza el código. El sistema tiene cinco tareas y sólo una llama a un modelo de lenguaje.

| TAREA                                                     | RESUELTA CON              | MOTIVO                                                                                   |
|-----------------------------------------------------------|---------------------------|------------------------------------------------------------------------------------------|
| Validar precios, horario y zona de entrega                | Código determinista       | Reglas exactas y auditables. Un modelo aquí sería más caro, más lento y menos confiable. |
| Numerar pedidos y reservas                                | Contador atómico en Redis | Requiere garantía de unicidad; un modelo no la da.                                       |
| Detectar duplicados                                       | Consulta a Notion         | Comparación exacta de teléfono y fecha.                                                  |
| Controlar la capacidad del turno                          | Suma aritmética           | El dato existe en la base; no hay nada que interpretar.                                  |
| Interpretar el comentario del cliente contra 23 políticas | Claude Opus 5             | Único punto con lenguaje natural ambiguo y consecuencias reales.                         |

#### 3.2 · Consumo medido

Medido sobre el prompt real en producción, con las 23 políticas activas:

| CONCEPTO                                               | CARACTERES | TOKENS APROX. |
|--------------------------------------------------------|------------|---------------|
| System prompt (instrucciones + 23 políticas)           | 5.324      | 1.520         |
| Mensaje de usuario (datos de la reserva)               | 210        | 60            |
| Definición del esquema de salida                       | 436        | 125           |
| Entrada total por reserva                              | 5.970      | ~1.700        |
| Salida (razonamiento + JSON), <code>effort: low</code> | —          | ~800          |

#### 3.3 · Matriz de decisión del modelo

| MODELO           | ENTRADA<br>USD/MTOK | SALIDA<br>USD/MTOK | COSTO<br>POR<br>RESERVA | 300<br>RESERVAS<br>AL MES | VEREDICTO PARA ESTA TAREA                                           |
|------------------|---------------------|--------------------|-------------------------|---------------------------|---------------------------------------------------------------------|
| Claude Opus 5    | 5,00                | 25,00              | \$0,0285                | \$8,55                    | Elegido. Distingue matices que cambian la decisión.                 |
| Claude Sonnet 5  | 3,00                | 15,00              | \$0,0171                | \$5,13                    | Alternativa razonable. Ahorra \$3,42 al mes: no compensa el riesgo. |
| Claude Haiku 4.5 | 1,00                | 5,00               | \$0,0057                | \$1,71                    | Sólo si el volumen creciera dos órdenes de magnitud.                |

**Por qué el modelo más capaz en la tarea más barata.** El costo total del componente de IA es de unos **8,55 USD al mes** con 300 reservas. Bajar a Haiku ahorraría 6,84 USD mensuales: menos de lo que cuesta un plato de la carta. Contra eso, un error de clasificación tiene consecuencias físicas — confirmar automáticamente una mesa que trae una mascota al salón, o no advertir una alergia grave, se paga con el cliente sentado en el restaurante.

La diferencia se observó en producción. Ante el comentario «*cumpleaños, llevamos nuestra torta y una botella de champagne, viene mi perro*», el modelo separó lo permitido de lo prohibido citando las políticas por nombre: la torta externa está permitida, el descorche no, y el perro se acepta en la terraza pero no en el salón. Esa granularidad es exactamente lo que los modelos más chicos aplanan.

3.4 · Optimizaciones aplicadas y descartadas

| TÉCNICA                            | ESTADO                | JUSTIFICACIÓN                                                                                                                                                                                                                                            |
|------------------------------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>effort: low</code>           | Aplicada              | Clasificar contra 23 políticas no exige razonamiento profundo. Reduce tokens de salida y latencia.                                                                                                                                                       |
| Tope de <code>max_tokens</code>    | Aplicada — 4.000      | Techo ante una respuesta anómala. No puede ser menor: el razonamiento sale del mismo presupuesto y con 1.000 la respuesta se cortaba.                                                                                                                    |
| Salida estructurada                | Aplicada              | Elimina los reintentos por JSON mal formado, que se pagan dos veces.                                                                                                                                                                                     |
| Filtrar políticas antes del prompt | Aplicada              | Sólo entran las marcadas <code>Activo</code> . Las inactivas no se pagan.                                                                                                                                                                                |
| Caché de prompt                    | Evaluada, no aplicada | El bloque de políticas es estable y supera el mínimo cacheable, pero las reservas llegan espaciadas y la ventana por defecto es de 5 minutos: casi todas las lecturas serían fallos de caché. Se vuelve rentable con TTL de una hora si el volumen sube. |
| API de lotes (50 % de descuento)   | Descartada            | Incompatible con el caso: el cliente espera respuesta en el momento y el encargado necesita el aviso al instante.                                                                                                                                        |

3.5 · Costo total de operación

| COMPONENTE                | USD / MES     | NOTA                            |
|---------------------------|---------------|---------------------------------|
| Anthropic — Claude Opus 5 | 8,55          | 300 reservas analizadas         |
| n8n Cloud                 | ~24,00        | plan de entrada                 |
| Vercel                    | 0,00          | plan gratuito                   |
| Notion                    | 0,00          | plan gratuito                   |
| Upstash Redis             | 0,00          | capa gratuita                   |
| Telegram                  | 0,00          | Bot API sin costo               |
| <b>Total</b>              | <b>~32,55</b> | <b>La IA representa el 26 %</b> |

## 4 · Seguridad y resiliencia

### 4.1 · Minimización de datos

| DATO                        | SE RECOGE | LLEGA AL MODELO | SE CONSERVA |
|-----------------------------|-----------|-----------------|-------------|
| Nombre                      | sí        | sí              | Notion      |
| Teléfono                    | sí        | <b>no</b>       | Notion      |
| Email                       | opcional  | <b>no</b>       | Notion      |
| Fecha, hora, personas, zona | sí        | sí              | Notion      |
| Comentario libre            | opcional  | sí              | Notion      |

El teléfono y el email **no se envían al modelo**: no aportan nada a la clasificación y son los datos más sensibles del conjunto. El registro de errores guarda un subconjunto todavía menor — número, fecha, personas y comentario — porque para depurar por qué falló un análisis no hace falta saber a quién llamar.

#### GESTIÓN DE CREDENCIALES

- Ningún secreto vive dentro del flujo: el del webhook está en una credencial Header Auth de n8n, y las claves de Notion, Anthropic, Gmail y Telegram en credenciales propias. El JSON que se publica en el repositorio no contiene ninguna.
- Los identificadores de bases, el plazo de vencimiento y el destinatario de los avisos se leen de variables de entorno con literal de respaldo, en un único nodo `Configuración`.
- Del lado de la aplicación, las variables viven en Vercel; el archivo local está excluido del control de versiones.

## 4.2 · Rutas de error

| FALLO                                                 | RESPUESTA DEL SISTEMA                                                                                                        | QUÉ VE EL USUARIO                             |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Datos incompletos en el formulario                    | HTTP 400 con la lista de campos faltantes, antes de tocar nada                                                               | Error en el formulario                        |
| Fuera del horario de atención o de la zona de reparto | HTTP 409 con el motivo                                                                                                       | Mensaje explicando cuándo o dónde sí se puede |
| Exceso de solicitudes desde una IP                    | HTTP 429 (20/hora por IP, 500/día global)                                                                                    | Aviso de reintentar más tarde                 |
| Notion no responde (pedidos)                          | HTTP 502; el pedido <b>no</b> se confirma                                                                                    | «No pudimos guardar tu pedido»                |
| n8n no responde (reservas)                            | HTTP 502; la reserva <b>no</b> se confirma                                                                                   | «No pudimos guardar tu reserva»               |
| <b>La API de IA falla o devuelve algo inesperado</b>  | <b>El flujo continúa: la reserva se marca para revisión humana y se escribe una fila en Errores , vinculada a la reserva</b> | <b>Nada: su reserva se registró igual</b>     |
| Gmail o Telegram fallan                               | El flujo continúa; la reserva ya está registrada                                                                             | Confirmación por teléfono                     |

**Criterio de diseño.** Falla ruidosamente lo que compromete el dato: si el registro no se pudo escribir, no se le confirma nada al cliente. Degrada en silencio lo accesorio: si el análisis o el aviso fallan, la reserva igual queda guardada y llega a un humano. Lo que degrada, deja rastro — de ahí la base **Errores** .

### GUARDAS EXPLÍCITAS

- **Bucle infinito:** el sondeo de aprobación se corta a las 4 horas marcando la reserva como **Vencida** y escalando por Telegram. El plazo es configurable.
- **Comparación de tipos:** los routers comparan booleano con booleano y número con número; ningún filtro compara una cadena contra un número.
- **Duplicados:** misma fecha y mismo teléfono con reserva ya confirmada se descarta sin crear una segunda.
- **Confianza en el cliente:** nula. Los precios se recalculan contra la carta real; un total manipulado desde el navegador se ignora.

## 4.3 · Punto de validación humana

El sistema **se detiene antes de confirmarle una mesa a un cliente** cuando se cumple cualquiera de estas condiciones:

| CONDICIÓN                                              | ORIGEN               |
|--------------------------------------------------------|----------------------|
| El comentario contradice una política vigente          | Análisis con RAG     |
| Menciona alergias, mascotas, accesibilidad o un evento | Análisis con RAG     |
| La confianza del análisis es menor a 0,7               | Umbral del router    |
| Más de 10 personas                                     | Regla de operación   |
| La ocupación del turno supera la capacidad             | Cálculo sobre Notion |
| El análisis de IA no se pudo completar                 | Ruta de error        |

En ese caso la reserva se guarda como `Esperando_Aprobacion` y el encargado recibe por Telegram el detalle, el motivo y el enlace directo a la ficha. Aprueba o rechaza **moviendo un campo en Notion** — sin aprender una herramienta nueva. El flujo lo detecta en cinco minutos y recién entonces contacta al cliente.

**Por qué evita el «efecto metralleta».** Ninguna comunicación con el cliente final ocurre en la rama de revisión hasta que un humano decide. El aviso al encargado no es una acción sobre el cliente: es la pregunta.

## 5 · Dashboard de control

Vista pública de Notion, en modo lectura, pensada para el celular del encargado durante el servicio.

| VISTA                | QUÉ MUESTRA                                   | PARA QUÉ                                     |
|----------------------|-----------------------------------------------|----------------------------------------------|
| Hoy                  | Reservas del día, ordenadas por hora          | Operación durante el servicio                |
| Esperando aprobación | Filtradas por Estado , con motivo y confianza | Cola de decisiones pendientes                |
| Por estado           | Tablero agrupado por Estado                   | Volumen y proporción automática vs. revisada |
| Ocupación            | Calendario por fecha                          | Carga por turno                              |
| Errores              | Base Errores con Resuelto sin marcar          | Tasa de fallos del sistema                   |

### INDICADORES

| INDICADOR              | CÁLCULO                                               | QUÉ SIGNIFICA SI SE MUEVE                                                              |
|------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------|
| Tasa de automatización | $\text{sin Requiere\_Revision} \div \text{total}$     | Si baja, las políticas se volvieron restrictivas o cambió el público                   |
| Tasa de errores        | $\text{filas en Errores} \div \text{reservas}$        | Si sube, hay un problema de integración, no de negocio                                 |
| Confianza media        | promedio de Confianza_IA                              | Una caída sostenida indica políticas desactualizadas frente a lo que la gente pregunta |
| Vencidas               | $\text{Estado} = \text{Vencida}$                      | Mide el tiempo de respuesta humano, no el del sistema                                  |
| Ocupación por turno    | $\Sigma \text{Personas} \div \text{Capacidad\_Total}$ | Saturación real del salón                                                              |

### MEDICIÓN DE REFERENCIA

Los mismos indicadores se calculan contra la API con `npm run notion:kpis` , lo que permite verificar que las vistas publicadas muestren lo correcto. Salida al cierre del desarrollo:

|                                 |       |        |
|---------------------------------|-------|--------|
| Reservas procesadas             | 20    |        |
| Resueltas automáticamente       | 7     | 35,0 % |
| Derivadas a revisión humana     | 13    | 65,0 % |
| Confirmada                      | 13    | 65,0 % |
| Rechazada                       | 5     | 25,0 % |
| Esperando_Aprobacion            | 2     | 10,0 % |
| Confianza media                 | 0,79  |        |
| Análisis con confianza < 0,7    | 5     |        |
| Análisis fallidos (confianza 0) | 1     |        |
| Errores registrados             | 0     |        |
| Tasa de error                   | 0,0 % |        |

La proporción de revisión humana es alta porque las pruebas se concentraron deliberadamente en casos conflictivos — mascotas, descorche, alergias, grupos grandes. Con reservas corrientes la relación se invierte.

**Antes de publicar la vista.** Una vista pública de Notion expone nombres, teléfonos y correos de clientes reales a cualquiera que tenga el enlace. Para la entrega conviene publicarla con datos de prueba, o esconder **Telefono** y **Email** : Notion respeta las propiedades ocultas en la versión publicada.

## 6 · Evidencia de pruebas

| # | ESCENARIO                                                  | RESULTADO ESPERADO                             | RESULTADO                                                           |
|---|------------------------------------------------------------|------------------------------------------------|---------------------------------------------------------------------|
| 1 | Reserva simple, 4 personas, sin observaciones              | Confirmación automática                        | <b>Confirmada</b> · confianza 1,00 · mail enviado                   |
| 2 | Comentario con torta propia, champagne y mascota           | Revisión humana citando políticas              | <b>Esperando_Aprobacion</b> · confianza 0,94 · 3 políticas citadas  |
| 3 | Reserva de 6 en un turno de capacidad 8, y luego otra de 4 | La segunda frenada por cupo                    | «Sin lugar: 6 + 4 supera 8» · ambas vinculadas a su turno           |
| 4 | Aprobación manual desde Notion                             | El sondeo detecta el cambio y avisa al cliente | Bucle cerrado · mail de confirmación recibido                       |
| 5 | <b>Camino infeliz:</b> formulario sin teléfono ni fecha    | Rechazo antes de tocar ninguna base            | <b>400 {"error":"invalido","details":["nombre","telefono",...]}</b> |
| 6 | <b>Camino infeliz:</b> webhook sin el secreto correcto     | Descarte sin ejecutar el flujo                 | Rechazado en el trigger                                             |
| 7 | <b>Camino infeliz:</b> fallo del análisis de IA            | La reserva sobrevive y queda registro          | Marcada para revisión · fila en <b>Errores</b> vinculada            |

### ARCHIVOS DE RESPALDO EN EL REPOSITORIO

- **n8n/reservas-arrecife.json** — flujo de reservas (28 nodos)
- **n8n/pedido-a-telegram.json** — flujo de pedidos
- **n8n/reservas/\*.js** — código de cada nodo, como archivos ejecutables y probables
- **notion/reservas-esquema.mjs** — el esquema de las bases, reproducible e idempotente
- **n8n/RESERVAS.md** — decisiones de implementación y sus motivos

**Sobre el modo de fallo dominante.** Los defectos más costosos del desarrollo no fueron caídas: fueron ejecuciones que n8n reportaba como exitosas haciendo menos de lo que declaraban. El análisis corría sin políticas, la ocupación se calculaba siempre en cero, el correo de confirmación nunca salía. Ninguno encendió una alarma. Por eso el sistema hoy registra *cuántas políticas consultó* cada análisis y deja constancia en **Errores** cuando degrada: la resiliencia sin observabilidad sólo esconde el problema.